

■ Demystifying AI

■ What's Actually Happening When You Talk to an LLM

Goal: Unbox the magic – mental models that help work smarter with AI every day.

■ Why This Talk?

AI tools are everywhere – Copilot, ChatGPT, Kiro, internal bots.

Most of us use them like a black box:

- Type something in → get something out
- Sometimes brilliant ✨ sometimes confidently wrong 🤖
- No idea why either happens

After this talk:

- Mental model of what's happening inside
- Write better prompts by understanding why they work
- Know when to trust and when to verify
- Use AI tools more effectively daily



■ Our Journey Today

■ Part 1: Unboxing the Magic

Under the hood

1. Predicting, not thinking
2. The meaning map & dimensions
3. Distance & cosine similarity
4. Tokenization
5. Text → numbers & context
6. Semantic search
7. Generation – one word at a time
8. Attention & temperature
9. Why it gets things wrong
10. Matrix math & cost

■ Part 2: Day-to-Day Skills

Practical application

11. Prompt engineering = writing a good brief
12. Context management
13. RAG – how AI uses internal docs

■ Part 3: Working Smarter

14. Skills, agents & system prompts
15. Testing AI features
16. Better inputs → better outputs



■ It's Not Thinking. It's Predicting.

The biggest misconception: "AI understands my question."

🗺 The Meaning Map (Embeddings)

- Words → coordinates on a map
- Similar meanings → nearby coordinates

```
"I love fries"      → 📍 (location A)
"Fries are great"   → 📍 (nearby!)
"The stock market" → 📍 (across town)
```

🗣 The Word Predictor (Generation)

- Constructs answers one word at a time
- "What's the most likely next word?"

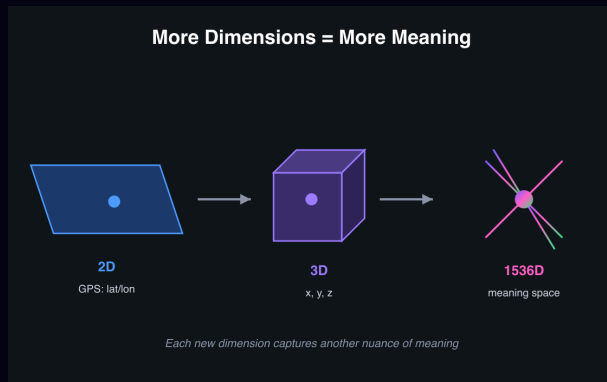
These two ideas power everything.



- Same neighborhood = similar meaning
- Different district = different meaning

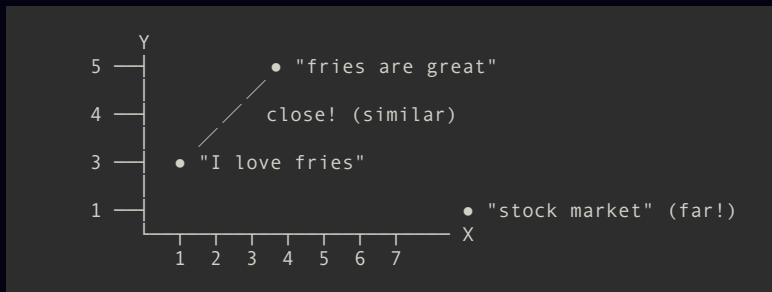
■ How Many Coordinates?

GPS: 2 numbers → locate a place Embeddings: 384-3072 numbers → locate a meaning



- More dimensions = more precise = better matching
- Like describing a person: height+weight (2) vs full profile (hundreds)

■ Measuring "Nearby" – Distance



- Close on the map = similar meaning
- Far apart = different meaning
- Same idea in 2D or 1536D – closer = more similar

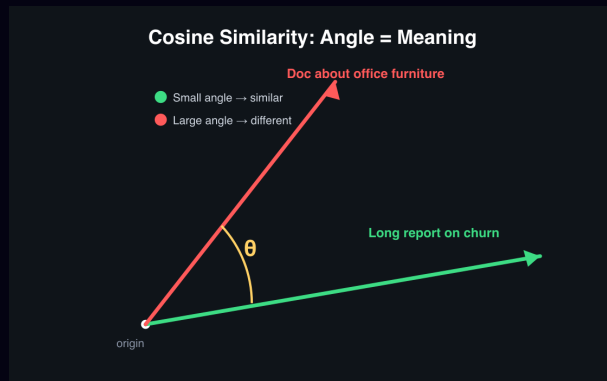
■ The Problem With Raw Distance

 10-page report on "customer churn" → long vector
 1-line Slack message: "users are churning" → short vector

Raw distance says far apart — one is just "bigger." But they mean the same thing!

Like two people pointing at the same star — one arm is longer.

→ Need a measure that ignores length, only cares about direction...



Angle	Cosine	Meaning
0°	1.0	Identical
30°	0.87	Very similar
60°	0.50	Somewhat related
90°	0.0	Unrelated

- Cancels out size → 10-page doc and 1-line message score the same
- Matches direction (meaning), not length

```
python scripts/similarity_demo.py
```

```
COSINE SIMILARITY – meaning direction, not length
```

```
"customer churn" vs "users cancelling" → 0.91 ✓
```

```
"customer churn" vs "french fries recipe" → 0.12 ✗
```

```
Same meaning = high score (close to 1.0)
```

```
Different meaning = low score (close to 0.0)
```

🎮 "What Goes Together?" game:

```
"cat" → meow, furry, pet, whiskers  
"dog" → bark, furry, pet, tail  
"car" → drive, road, fast, wheels
```

- cat & dog share context (furry, pet) → similar numbers
- cat & car share nothing → different numbers

Word2Vec (2013): Words in similar contexts get similar coordinates.

"King" - "Man" + "Woman" = "Queen" ← the math actually works!

Modern models add context on top:

- Same word "love" gets different numbers in "love fries" vs "love letter"

■ Context Changes the Numbers

Same word → different numbers depending on surroundings:

```
"I love fries"      → "love" = food/enjoyment  
"I love coding"    → "love" = passion/tech  
"love letter"      → "love" = romance
```

Same spelling, completely different meanings:

```
"The bank was steep"      → riverbank  
"The bank was closed"    → financial institution  
  
"She saw a bat"          → animal or sports equipment?  
  
"Apple released a new product" → tech company  
"Apple released a sweet aroma" → fruit
```

This is why AI handles ambiguity well – context refines the numbers.

LLMs don't read words. They read tokens – chunks of text.

What it seems like	What actually happens
"Summarize this 10-page doc"	~4,000 tokens of input
"Context window: 128K tokens"	≈ 96,000 words ≈ 300-page book
"Why did it cut off mid-sentence?"	Hit the max output token limit
"Why does it cost more for long prompts?"	Billed per token (input AND output)

Rule of thumb: 1 token ≈ ¾ of a word.

This is why AI has a memory limit – measured in tokens, not words.

```
python scripts/tokenization_demo.py
```

```
TOKENIZATION – How AI reads text
```

```
"I love french fries"
```

```
→ [I] [love] [french] [fries] = 4 tokens
```

```
"Unbelievable!"
```

```
→ [Un][believ][able][!] = 4 tokens
```

```
Rule of thumb: 100 words ≈ 133 tokens
```

■ How Semantic Search Works

Keyword search: "fries" → only finds docs containing "fries" Semantic search: "fries" → also finds "crispy potato snacks"

Query: "Why are users dropping off during checkout?"
↓ (drop a pin on the meaning map)

- ✅ "Cart abandonment analysis Q4" (nearby!)
- ✅ "Reducing friction in purchase flow" (nearby!)
- ❌ "How to fry an egg" (across town)

Found "cart abandonment" without saying those words. Meaning > keywords.



⚠ Trust the structure, verify the facts.

■ How LLMs Generate Text

One word at a time – a probability game:

Input: "The capital of France is"

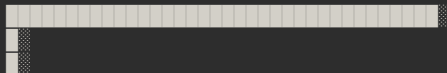
Paris → 94.2%

a → 1.8%

located → 1.1%

banana → 0.0%

Picks: "Paris"



Then feeds it back and predicts the next word. And the next. And the next.

That "typing" effect isn't flair – it's literally generating one token at a time.

```
python scripts/next_word_demo.py
```

Shows:

- Probability distribution as ASCII bar chart
- Temperature = 0 → always picks "Paris" (deterministic)
- Temperature = 0.7 → usually "Paris", sometimes "Lyon"
- Temperature = 1.5 → creative/chaotic picks

"As a QA engineer, write test cases for the LOGIN page
focusing on SECURITY edge cases"

As a QA engineer write test cases for the LOGIN page
focusing on SECURITY edge cases
■ = high attention _ = low attention

Specific, keyword-rich prompts → higher attention on what matters.

```
python scripts/attention_demo.py
```

Shows each word colored by attention weight:

-  Green = high focus (SECURITY, LOGIN, test, QA)
-  Dim = low focus (a, the, for, on)

Under the Hood – Why Matrices?

Each word = a row of numbers. The AI processes ALL words at once.

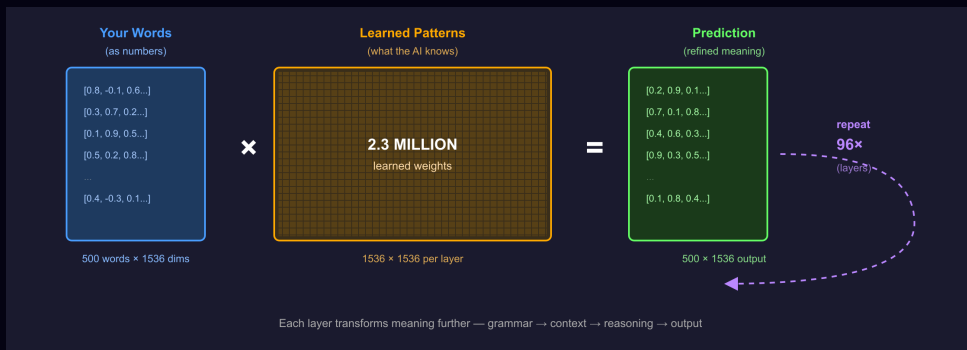
```
"Write" → [0.4, 0.1, 0.8, ...]  
"test"  → [0.3, 0.7, 0.2, ...]  
"cases" → [0.1, 0.9, 0.5, ...]
```

← this IS a matrix

To figure out distance between meanings (like $y = mx + c$, but for similarity):

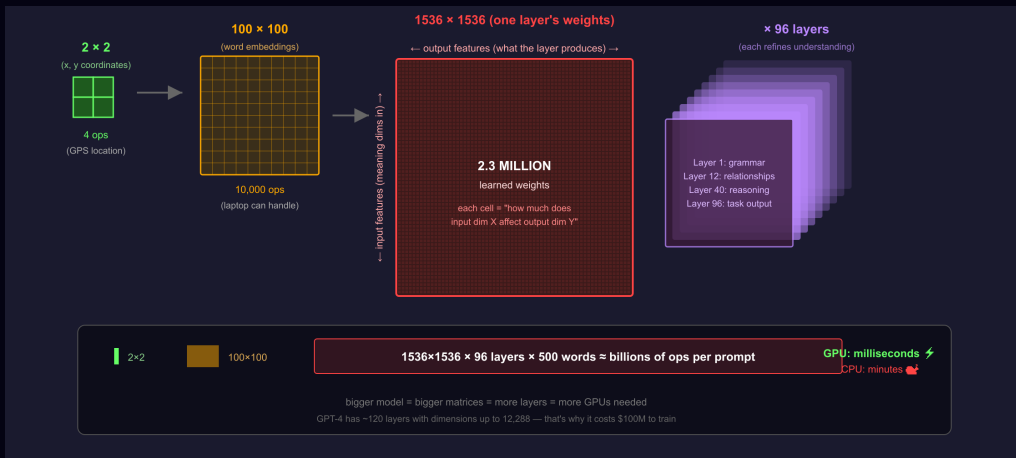
$$d = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$$

Doing this for every word pair, across 1536 dimensions, across 96 layers = trillions of operations. Matrices let GPUs do it all in parallel.



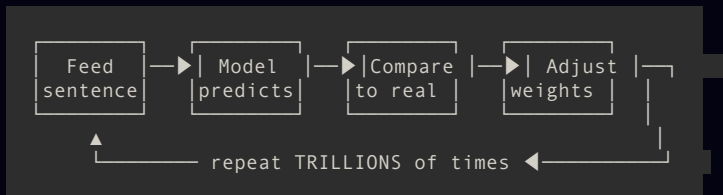
■ The Complexity Explosion – Why GPUs?

"customer churn" → [0.82, -0.14, 0.67, 0.03, -0.91, 0.45, ... ×1536 numbers]
"users leaving" → [0.79, -0.11, 0.63, 0.05, -0.88, 0.42, ... ×1536 numbers]



■ Why LLMs Cost So Much

Training = adjusting all weights until predictions improve:

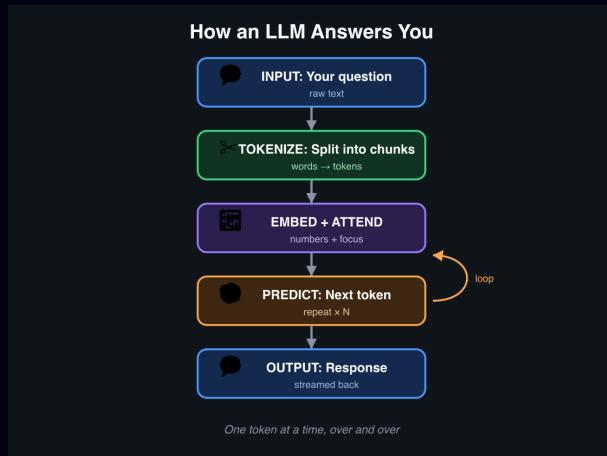


What	GPT-4 Scale
Parameters (weights to tune)	~1.8 trillion
GPUs (A100s, in parallel)	~25,000
Training duration	~90-100 days non-stop
Compute cost	\$78-100+ million

Sources: OpenAI CEO confirmed >\$100M; semi-analysis estimates

- API calls cost money → renting time on these GPU clusters
- Fine-tuning = adjusting an already-trained model → much cheaper
- 7B model fits on a laptop; 400B+ needs a data center





- Typing effect = generating one token at a time
- Each step is independent → errors compound over long outputs

■ Part 2: Day-to-Day Skills

■ Practical application of what was just covered

From mental models to daily workflow

■ Prompt Engineering = Writing a Good Brief

Bad brief to a designer: "Make it look nice." Good brief: "Hero banner, blue tones, mobile-first, CTA button."

✗ Vague:

```
Write test cases for login.
```

✓ Structured:

```
Role: QA engineer
Context: OAuth2 + email/password,
        2FA enabled, 3-strike logout
Task: 10 test cases covering
      happy path, edge cases, security
Format: table [ID, Scenario,
              Steps, Expected Result]
```

The four levers:

Lever	What it does
Role	Sets perspective – "senior QA engineer"
Context	Relevant info – paste the spec, API contract
Task	Exactly what's needed – "write edge case tests"
Format	Output shape – "markdown table"

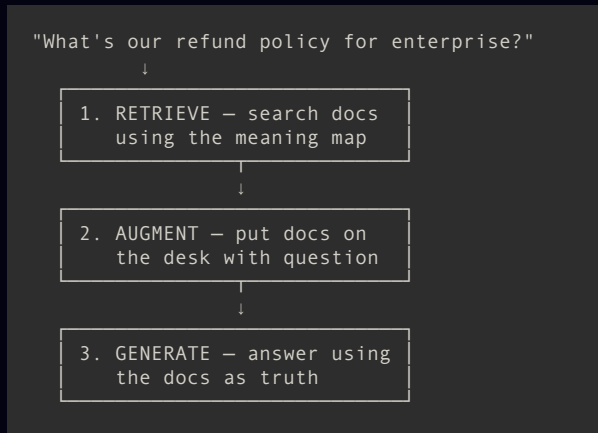
■ Context Management – The Hidden Skill

AI memory = a desk. Only sees what's on it right now.



- Start fresh for new topics
- Paste context explicitly – it doesn't "remember"
- Be selective – relevant file, not entire codebase
- Summarize long chats → start new thread

Problem: AI doesn't know internal docs. Solution: Fetch first, then ask. (Retrieval Augmented Generation)



Garbage docs in → garbage answers out.

■ Part 3: Working Smarter with AI

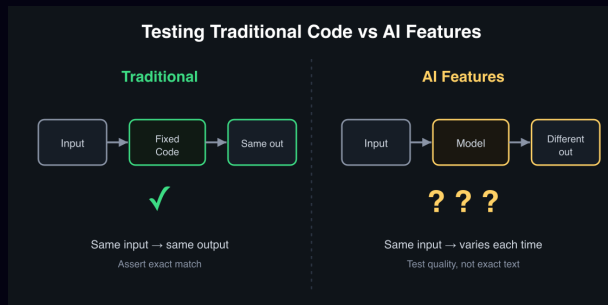
■ Practical patterns for teams

Skills, testing, and getting better
outputs



Concept	Analogy
Skills	Recipes – pre-built task instructions
Agents	Assistants – role + tools + boundaries
System Prompt	Job description – shapes behavior

Better job description → better assistant.



Traditional

Same input → same output
Exact-match assertions
Code change = behavior change

AI Features

Same input → different output
"Good enough" evaluation
Model update = silent shift

- 🔄 Regression without code changes – keep golden test sets
- ⚖️ Bias – test diverse inputs
- 🧐 Confidence ≠ correctness – verify with known answers

✗ "As a user, I want to log in"

✓ "Enterprise user, SSO via SAML 2.0,
fallback to email/password if IdP down,
30 min session timeout on inactivity"

✗ "Login doesn't work sometimes"

✓ "Login 401 when SSO token expires during 2FA.
5 min idle. Chrome 124. Staging.
SSO → wait 5 min → enter code → 401"

AI output quality $\propto$ input quality.

■ Security – What NOT to Paste

Don't paste	Why
Passwords, API keys, tokens	Could be logged or leaked
Customer PII	Privacy & compliance risk
Internal credentials	Data residency concerns
Proprietary algorithms	IP protection

Key question: Where does the prompt go?

- Cloud AI (ChatGPT, Claude) → external servers
- Enterprise AI (internal tools) → within the org
- Local models → never leaves the machine

Know the org policy before pasting anything sensitive.

■ When to Trust AI Output

Scenario	Trust	Action
Brainstorming	✓ High	Use freely
Drafting docs/emails	✓ High	Light review
Generating test cases	● Medium	Check completeness
Summarizing docs	● Medium	Spot-check facts
Factual claims/numbers	● Low	Always verify
Security/compliance	● Low	Expert review
Legal/medical/financial	⊘ Don't	Human experts only

AI is a draft generator, not a decision maker.

■ Key Takeaways

1. It's a meaning map, not magic. Text → coordinates. Search finds neighbors.
2. Context is everything. What's on the "desk" determines answer quality.
3. Prompt engineering = good brief. Role + context + task + format. Specificity wins.
4. RAG = fetch first, then ask. Good docs → good answers.
5. Better inputs = better outputs. Structured specs and clear context make AI dramatically more useful.

■ The End

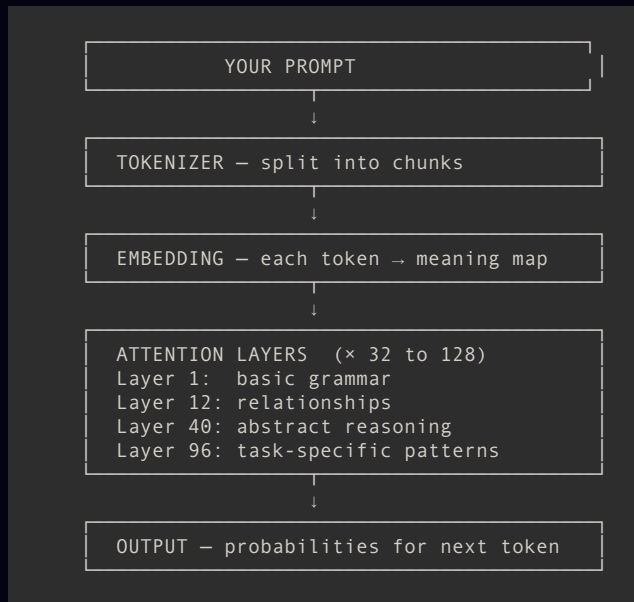
AI isn't magic. Understanding it is a superpower. 🚀

Questions?

📧 jeevan.dc24@alumni.iimb.ac.in

🌐 noobj.me





More layers = deeper understanding. GPT-4 has ~120 layers.

■ Appendix: Model Sizes

B = Billions of parameters (learned "knobs")

```
Small (7B)      → well-read college student
Medium (30B)    → subject matter expert
Large (70B)     → senior consultant
Frontier (400B+) → team of senior consultants
```

More parameters	Trade-off
Better nuance & reasoning	Slower responses
Fewer hallucinations	More expensive per token
Better at complex instructions	Needs more hardware

Match the model to the job – quick tasks → small model, complex tasks → large model.

The Reviewer:

```
Role: [role]. Review [artifact] for [criteria].  
Flag as [Critical/Major/Minor]. Format: table.
```

The Generator:

```
Given [context], generate [N] [things] covering [categories].  
Include edge cases. Format: [structure].  
Example: [one example]
```

The Analyzer:

```
Analyze [data]. Summarize findings.  
Top 3 issues. Next steps.  
Reference specifics.
```

Term	Plain English
LLM	Large Language Model – the AI brain
Embedding	Text → point on the meaning map
Cosine similarity	Closeness by direction, not distance
Token	A chunk of text (~¾ of a word)
Attention	How AI decides which words matter
Temperature	Creativity dial (low=safe, high=creative)
Parameters	The "knobs" a model learned
Semantic search	Finding by meaning, not keywords
RAG	Fetch docs, then ask
Context window	How much AI can see at once
Hallucination	Confidently making something up
System prompt	The job description for an AI
Agent	AI with a role, tools, and boundaries